# Neural Networks Dismantled

A Unified Framework for Semantics, Mathematics, and Execution

*Second Edition Draft — 2026*

**Book Promise**
After reading this book, a beginner should be able to look at any unfamiliar neural-network architecture and ask the right questions: What are the entities? How do they interact? What trains them? How does hardware execute it? And — crucially — *why was it designed this way?*

# Contents

Chapter 0

# Preface: Scope, Assumptions, and What This Book Is Not

This book is an orientation layer. It builds the bridge between a machine-learning textbook, a linear algebra course, and a GPU programming manual — without trying to replace any of them.

It covers supervised and self-supervised deep learning: how architectures are designed, why they are designed that way, and how they execute on hardware. It does not cover reinforcement learning in depth, generative adversarial networks, probabilistic graphical models, classical machine learning, or the pre-deep-learning era. Each deserves its own book.

The assumed reader knows what a function is and has seen a matrix. No calculus or coding experience is required, though both help you go further afterward.

## What This Edition Adds

- **Micro-walkthroughs** in every major architecture chapter: concrete symbol traces showing representations evolving step by step.
- **Tensor Trace boxes**: shape evolution through a full forward pass.
- **Tokenization** as a dedicated chapter: the gap between raw text and model input.
- **Sampling and decoding**: how logits become generated text.
- **Failure Mode callouts**: what breaks, and why, in each architecture.
- **Test Your Mental Model prompts**: reflection questions after dense sections.
- Precision fixes throughout: normalization, distance-similarity, attention-head claims.

PART 1

# The Orientation

Why the field feels difficult, and the two lenses that make it navigable.

Chapter 1: The Transparency Gap • Chapter 2: The Three-Layer Map

Chapter 1

# The Transparency Gap

## A Concrete Starting Point

You open an app and type: *Explain black holes in simple terms.* In the next few seconds, something specific happens. Your text is split into fragments called **tokens**. Each token is converted to an integer. That integer indexes into a learned table, retrieving a vector of several thousand decimal values — its **embedding**. Those vectors flow through dozens of identical processing stages. In each stage, every token consults every other token and updates its own representation. After the final stage, one transformation produces a probability distribution across the model's entire vocabulary. The highest-probability token is chosen, appended, and the process repeats until the response is complete.

Almost every phrase in that description — tokens, embeddings, attention, representations, vocabulary distribution — you will understand precisely by the last page. The gap between that description and your current understanding is the **Transparency Gap**: the distance between code we write, ideas we intend, and physical computation that executes.

## Why the Gap Exists

Neural-network terminology grew historically, not systematically. The same object — a vector representation of a data unit — is called a hidden state in an RNN, a feature map in a CNN, a node embedding in a GNN, and an activation in a runtime trace. These are not different things. They are different names for the same idea, coined by different communities for different problems, and never reconciled.

**Key Concept**
Our goal is **disentanglement**: separating the **What** (information being represented), the **How** (math implementing it), and the **Where** (hardware executing it). When these three are tangled, the field feels like a fog. Separated, its underlying simplicity emerges.

## Four Misconceptions to Clear Now

**Common Misconception**
**Neurons are like brain neurons.** They are not. The analogy is historical, useful in 1943, not functionally accurate today. A neural network neuron is a weighted sum followed by a nonlinear function. Releasing the brain metaphor makes the math dramatically clearer.

**Common Misconception**
**The model "understands" things.** The model's internal mechanisms are statistical and representational rather than human-cognitive. It has learned regularities in a high-dimensional space that produce useful outputs. This is remarkable. It is not the same as understanding, and conflating them causes confusion when models fail in unexpected ways.

**Common Misconception**
**Training is programming.** Programming tells a system exactly what to do. Training applies pressure through a loss function; useful structure emerges from that pressure. The resulting behaviour is encoded in billions of numeric parameters, not in readable logic.

**Common Misconception**
**Attention means the model pays attention like a human.** Attention in neural networks is weighted aggregation: each entity computes a weighted sum over others' values, where weights are learned to be useful. The word is a suggestive metaphor, not a cognitive description.

## The Disentanglement Discipline

For any concept, answer three questions, each in one sentence: **What** information is being represented or moved? **How** does a mathematical function or tensor implement it? **Where** does hardware physically execute the computation? The next chapter makes each question precise.

Chapter 2

# The Three-Layer Map: What, How, Where

Every neural-network concept lives at three levels simultaneously. Confusion comes from mixing them. The three-layer map tells you exactly which level you are at.

| Layer | Question Answered | Attention Example |
|---|---|---|
| What (Semantic) | What information is represented or moved? | A token gathers relevant information from other tokens |
| How (Mathematical) | What function or tensor implements it? | softmax(QK■ / √d) V |
| Where (Execution) | How does hardware physically compute it? | matmul → scale → mask → softmax → matmul |

## The Three-Sentence Exercise

**Key Concept**
**When any concept confuses you:** write three sentences — one at each level. If you cannot write all three, you have found precisely where your understanding breaks down. This is the primary diagnostic tool of the book.

## Example: Convolution

| Level | Description |
|---|---|
| What | A spatial location uses evidence from its local neighbourhood. |
| How | A shared weight kernel computes a weighted sum over a local patch. |
| Where | The runtime slides patches into SIMD-friendly memory access patterns. |

## Example: A Dense Layer

| Level | Description |
|---|---|
| What | A representation is projected into a new coordinate space. |
| How | y = xW + b, matrix multiplication plus bias addition. |
| Where | A BLAS GEMM call on CPU or a tensor core operation on GPU. |

**Test Your Mental Model**
Pick one concept you already know — ReLU, dropout, softmax. Write one sentence at each level. Which level was hardest to articulate?

PART 2

# Foundations: The Anatomy of a Transformation

The vocabulary every architecture is built from.

Chapters 3 – 11

Chapter 3

# The Universal Neural-Network Loop

Every architecture follows the same repeating cycle. Understanding the loop means every specific architecture you encounter is a specialisation of something already known.

```
raw input → encode into initial representations → repeated blocks of: interact + update →
task-specific readout → loss or output
```

## The Generic Formula

```
H_0 = Encode(input) for l in 0..L-1: M_l = Interact(H_l, structure) H_l+1 = Update(H_l, M_l) Output
= Readout(H_L)
```

This is a template. Transformers, CNNs, RNNs, and GNNs all fit it if you choose the right definitions of Encode, Interact, Update, and Readout.

## The Five Primitive Questions

- What are the entities?
- What vector or tensor represents each entity?
- What structure constrains interaction: sequence, grid, graph, set, or none?
- How do entities exchange information?
- How is the final output extracted and how is the system trained?

**Key Concept**
We do not calculate an answer directly. We evolve a signal through stages of refinement until it is in the right form for the task. Architecture choice is primarily a choice about how to structure that refinement.

Chapter 4

# Tokenization: How Raw Text Becomes Entities

Tokens are the entities of language models. Every subsequent chapter that discusses language assumes you understand what a token is and why tokenization is non-trivial. This chapter fills that gap.

## The Problem: Computers Operate on Numbers

A language model cannot process the string "cat" directly. It needs an integer it can use to look up a vector in a learned table. Tokenization is the procedure that converts raw text into a sequence of such integers. The design of this procedure has significant downstream effects on what the model can and cannot do.

## Three Approaches, Increasing Sophistication

| Approach | Unit | Vocabulary Size | Problem |
|---|---|---|---|
| Character-level | Each character | ~100 | Very long sequences; no morphological sharing |
| Word-level | Each word | 50,000–500,000 | Unknown words; vocabulary explosion across languages |
| Subword (BPE) | Frequent character groups | 30,000–64,000 | Balances sequence length against coverage |

## Byte Pair Encoding (BPE)

BPE is the most widely used subword method. It starts with individual characters and iteratively merges the most frequent adjacent pair into a new token. After enough merges, common words like 'the' become single tokens, while rare words like 'unbelievableness' are split into subword pieces: [un][believ][able][ness]. The vocabulary — the set of all valid tokens — is fixed at training time.

## Why This Matters: Surprising Consequences

- **LLMs count letters badly.** Ask a model how many r's are in 'strawberry.' It may fail because 'strawberry' might be a single token, making character-level introspection unreliable.
- **Numbers are fragile.** "9.11" and "9.9" may tokenize to different numbers of tokens, making numeric comparison awkward for a model that learned from token sequences rather than numeric values.
- **Multilingual models are unequal.** The same sentence in English might be 10 tokens while an equivalent in another language is 40, costing 4x the context window and compute.
- **Vocabulary size is a hyperparameter.** Larger vocabularies reduce sequence length but increase embedding table size.

**Tensor Trace: Text to Token IDs**
Input string: "The cat sat" Tokenizer splits: ['The', ' cat', ' sat'] Vocabulary lookup: [464, 3797, 3031] Shape entering the model: [S=3] — one integer per token After batch dimension: [B=1, S=3] This integer sequence is the input to the embedding layer.

**Failure Mode**
Because vocabulary is fixed at training time, any token outside the vocabulary must be split into subword fallbacks or handled with an unknown token. This is why models can behave erratically on novel technical terms, non-standard spellings, or newly coined proper nouns.

**Test Your Mental Model**
If a model's vocabulary contains 'ing' as a token, and you ask it to process 'running,' what are the possible tokenizations? Why might different tokenizations of the same word cause problems for the model?

Chapter 5

# Tensors and Shapes: The Semantic Organizer

A tensor is commonly taught as a multidimensional array. That is a *Where*-level definition. At the *What* level, a tensor is a **Semantic Organizer**: its axes encode information categories, not just dimensions.

| Level | Description |
|---|---|
| What | Axes represent semantic categories: entities, features, time, examples. |
| How | A multilinear map over a product of vector spaces. |
| Where | A contiguous typed memory block with a stride array for hardware traversal. |

## The Anatomy of Shape: [B, S, F]

A tensor of shape [32, 512, 768] carries three semantic clauses:

- **Batch (32):** 32 independent examples processed together for hardware efficiency. They never interact. This dimension exists for the GPU, not the model logic.
- **Sequence (512):** 512 ordered entities per example. Temporal or positional structure lives here.
- **Features (768):** Each entity is described by 768 numbers — the width of the model's representational workspace per entity.

**Key Concept**
Batch is for the GPU. Sequence, grid, and graph axes belong to the model. When reasoning about what a model does to one example, mentally remove the batch axis.

## Strides and Views

Every tensor has shape, dtype, device, and stride. **Stride** specifies the memory jump per step along each axis. A **view** reinterprets memory without copying. Operations like transpose and reshape are often free metadata changes, but some kernels require contiguous memory and force a copy.

**Test Your Mental Model**
A tensor of shape [B, S, F] is transposed to [B, F, S]. Which semantic axis now runs fastest in memory? When would this matter for performance?

Chapter 6

# Representations, Embeddings, and Latent Spaces

## Representation Learning: The Unifying Idea

All architectures in this book are doing the same thing at the deepest level: **representation learning**. They learn to map inputs into geometric spaces where the task becomes computationally easy. This is why pre-trained representations transfer to new tasks, why scale helps, and why architecture choice matters — different structures induce different geometric biases.

## Embedding vs Representation vs Hidden State

| Term | Definition | Example |
|---|---|---|
| Embedding | Initial vector for a raw object, before context processing | Token id 464 → 768-dim vector |
| Representation | Any internal vector at any processing stage | Layer 12 hidden state |
| Activation | A computed value during the forward pass | Output of a GELU function |
| Latent space | The coordinate system in which representations live | Semantic geometry of hidden states |

## Geometric Proximity and Semantic Similarity

In a well-trained latent space, geometric proximity **often correlates with** semantic similarity. Representations of related concepts tend to cluster; unrelated concepts tend to be distant. This geometry is not programmed — it emerges from training pressure. However, embedding spaces are anisotropic and metric-sensitive: the relationship between distance and similarity is useful and approximate, not universal.

> **You've Already Used This**
> Every semantic search engine, content recommendation system, and document retrieval pipeline uses embeddings. Your query becomes a vector; items nearby in that space are returned. The geometry of the latent space is the product.

## Context Changes Representations

The token 'bank' in 'the bank approved the loan' and in 'the river overflowed the bank' begins with the same embedding. After attention layers, its hidden state diverges: one representation has moved toward financial concepts; the other toward geographic concepts. The embedding represents identity; the hidden state represents meaning in context.

Chapter 7

# Layers as Learned Transformations

A layer is a learned transformation: it takes a representation and returns a new one. A deep network composes these: x_0 → x_1 → … → x_L.

## Depth, Width, and Nonlinearity

- **Depth** is the number of transformation steps. Early layers detect simple patterns; later layers compose them into complex ones.
- **Width** is the representation dimensionality at each step. Wider networks have more representational capacity per entity, at greater memory and compute cost.
- **Nonlinearity** is essential. Without it, all layers collapse to a single linear map regardless of depth. Nonlinear activations allow conditional features: patterns present only when specific input conditions are met.

## Residual Connections

A residual connection computes **x + update(x)** rather than replacing x. This turns each layer from a replacement into a refinement. Crucially, gradients can flow directly through the additive skip, bypassing any problematic layer transformations. This is what enables 50-, 100-, or 1000-layer networks to train.

## Normalization

Layer normalization rescales each entity's vector to approximately zero mean and unit variance. Without it, representations in deep networks tend to explode or vanish across layers, making gradient-based learning unstable. Normalization is primarily an optimization and stability mechanism rather than a semantic objective. Its purpose is not to inject new information, though rescaling can influence representational geometry and learning dynamics.

**Failure Mode**
A network without normalization in a deep architecture will often either diverge (exploding gradients) or stall (vanishing gradients) within the first few thousand training steps. This is not a data problem or a learning-rate problem — it is a signal that the representational scale is unconstrained.

Chapter 8

# Training: Loss, Gradients, and Optimization

## Loss

The loss function is the signal that judges outputs and creates training pressure. The model is not told to learn concepts — it is told to reduce a number. Useful internal structure emerges because it helps reduce that number.

**Key Concept**
Loss defines the pressure. Architecture defines the structure within which the model responds. Data defines what patterns are available to learn. All three determine the representations that emerge.

| Task | Typical Loss | Representational Pressure |
|---|---|---|
| Language modeling | Next-token cross-entropy | Compress context into representations that predict the next word |
| Classification | Cross-entropy | Separate class clusters in latent space |
| Regression | Mean squared error | Produce scalar outputs near targets |
| Contrastive | Similarity loss | Pull related pairs together; push unrelated apart |
| Diffusion | Denoising loss | Learn to reverse a corruption process |

## Gradients and Optimization

A **gradient** tells how a small change in a parameter would change the loss. The **Adam optimizer**, the standard choice, maintains adaptive per-parameter estimates of gradient magnitude, making training robust across many different problem scales and architectures.

## Generalization

The goal is not to memorize training examples but to learn transformations that work on unseen examples from the same distribution. Generalization depends on data diversity, architecture inductive bias, objective design, scale, and regularization.

Chapter 9

# Autograd and the Backward Graph

Autograd is automatic differentiation. It tracks every operation in the forward pass and automatically applies the chain rule to compute gradients for every parameter.

## A Scalar Worked Example

Consider the simplest possible model: one parameter w, one input x, one target y_true.

```
x = 2.0, w = 3.0, y_true = 4.0 # Forward pass prediction = x * w # = 6.0 loss = (prediction - y_true)**2 # = (6-4)^2 = 4.0 # Backward pass (chain rule) d_loss/d_prediction = 2 * (prediction - y_true) # = 2*(6-4) = 4.0 d_loss/dw = d_loss/d_prediction * d_prediction/dw = 4.0 * x = 4.0 * 2.0 = 8.0 # Parameter update (learning rate = 0.1) w_new = w - 0.1 * 8.0 # = 3.0 - 0.8 = 2.2 # New prediction (better) prediction_new = x * w_new = 2.0 * 2.2 = 4.4 # closer to 4.0
```

Every gradient in a real neural network — across billions of parameters — is computed by this same chain rule, applied automatically by the autograd system.

## Forward and Backward Graphs

```
Forward: x, W, b → xW → xW+b → prediction → loss Backward: loss.grad=1 → d(prediction) → dW, db, dx
```

The backward graph runs in reverse. Each operation records how to propagate gradients through itself — this is stored during the forward pass so it is available when backpropagation runs.

**In Practice**
Autograd has significant memory cost. During training, intermediate activations must be saved for backward. Training typically uses 2–4× the memory of inference on the same batch. When a training run exceeds GPU memory, reducing batch size or using gradient checkpointing are the standard responses.

**Test Your Mental Model**
In the scalar example above, if x were 4.0 instead of 2.0 (everything else the same), what would the gradient dw be? Would the parameter update be larger or smaller? What does this tell you about how input scale affects training?

Chapter 10

# Training vs Inference: Two Operating Regimes

The same model behaves very differently during training and inference. Conflating them is one of the most persistent sources of beginner confusion.

| Property | Training | Inference |
|---|---|---|
| Input visibility | Full sequence, processed in parallel | One token at a time (autoregressive) or full input (encoder) |
| Computation | Forward + backward pass | Forward pass only |
| Gradients | Tracked | Not computed |
| Memory | Activations + gradients + optimizer state | Activations only |
| Parallelism | High — all positions at once | Limited for generation |
| KV cache | Not used | Used — caches prior keys and values |
| Batch size | Large | Variable, often 1 |

## The KV Cache

Each transformer layer computes keys and values from its input. During generation, prior tokens' keys and values are stable — they will not change. The **KV cache** stores them so the model computes only the query for the new token and attends over the cache. Without it, each generation step costs O(n) in the number of prior tokens, because everything would be recomputed from scratch.

**You've Already Used This**
When a language model generates a response and you see tokens stream out at roughly constant speed, that's the KV cache at work. The first token takes slightly longer (full forward pass); subsequent tokens are faster (one cached step each).

**Failure Mode**
At long context lengths, the KV cache itself becomes the memory bottleneck. For a model with 32 layers, 32 heads, and a 128k token context, the KV cache can exceed the memory cost of the model weights themselves. This is why long-context inference is an active engineering problem.

Chapter 11

# Data and Representations: What Training Data Does

Architecture and objective receive most of the attention in deep learning. Data is often treated as a given. This is a mistake. Data is the primary determinant of what representations a model can possibly learn.

**Key Concept**
Architecture defines the structure within which representations are learned. Data defines which representations it is possible to learn. Both matter; neither substitutes for the other.

## Data Determines the Possible

A model can only learn patterns present and consistent in its training data. If a concept appears rarely, its representation will be shallow. If a language appears in 1% of the training corpus, the model will be substantially weaker in that language than in one representing 30% of the corpus. These are data problems, not architecture problems.

## Distribution Shift

A model trained on one data distribution and evaluated on a different one will fail in proportion to how different those distributions are. A medical classifier trained on one hospital's equipment may fail on another's. A language model trained on text from 2024 will handle post-2024 events poorly. Distribution shift is the central challenge of deploying models reliably in the real world.

## Pre-Training Data Composition

For large models, data composition deliberately shapes representational strengths. More code in training data produces stronger code reasoning. More scientific text produces better scientific concept representations. These are engineering decisions with large, measurable effects.

## Quality vs Quantity

For a fixed compute budget, smaller quantities of high-quality, diverse data often produce better representations than larger quantities of noisy or repetitive data. Filtering, deduplication, and curation are active engineering disciplines.

**In Practice**
When a model behaves unexpectedly, ask two questions before touching the architecture: (1) Is this behaviour explained by something in the training distribution? (2) Would the model have seen enough examples of this type to learn a reliable representation?

PART 3

# The Framework

The two analytical tools that unlock any architecture.

Chapter 12: EIU Framework • Chapter 13: The Structure Spectrum

Chapter 12

# The Entity-Interaction-Update Framework

This is the hinge of the book. Three questions decode any layer in any architecture.

| Term | Definition | Examples |
|---|---|---|
| Entity | The unit being represented | Token, patch, graph node, pixel region, timestep |
| Representation | The vector or tensor state of that entity | Hidden vector, channel vector, node embedding |
| Structure | Which entities can interact, and how constrained | Sequence, grid, graph, set, memory |
| Interaction | How information moves between entities | Attention, convolution, message passing, recurrence |
| Update | How representation changes after interaction | MLP, activation, normalization, residual addition |
| Readout | How the output is extracted | Classifier head, logit projection, pooling |
| Objective | What training pressure is applied | Cross-entropy, denoising loss, contrastive loss |

**Key Concept**
**One-sentence architecture test:** If you cannot state what the entities are and how they interact, you do not yet understand the architecture. Everything else is detail layered on top of this.

## EIU Applied

Using EIU: a **transformer** has entities = tokens, interaction = each token attends to all others weighted by learned relevance, update = per-token MLP with residual. A **CNN** has entities = spatial locations, interaction = local convolution with shared weights, update = nonlinearity and normalization. A **GNN** has entities = nodes, interaction = message passing from neighbors, update = node MLP.

**Test Your Mental Model**
Apply EIU to an LSTM. What is the entity? What is the interaction? What is the update? How does your answer change if you consider the gating mechanism specifically?

Chapter 13

# The Structure Spectrum: From Dense to Explicit

Interaction rules form a continuous spectrum. Seeing them this way makes hybrid and novel architectures immediately interpretable.

| Structure | Who Communicates | Interaction Style | Example |
|---|---|---|---|
| No structure (set) | Everyone to everyone | Dense / fully learned | Bidirectional attention |
| Sequence (causal) | Each position to earlier positions | Causal masking | Autoregressive LM |
| Local sequence | Position to nearby neighbours | Windowed | 1D CNN, sliding window attention |
| Grid | Location to spatial neighbours | Fixed local kernel | 2D CNN |
| Explicit graph | Node to connected neighbours | Edge-defined | GNN |
| Hierarchical | Fine entities then pooled | Coarsening | U-Net, pooling layers |
| Cross-modal | Two pools attend to each other | Cross-attention | Encoder-decoder |

## Inductive Bias as a Position on the Spectrum

An **inductive bias** is a structural assumption that makes some patterns easier to learn than others. CNNs have a strong bias toward translation equivariance. Transformers have weak spatial bias and must learn spatial structure from data. When data is plentiful, weak inductive bias often wins. When data is scarce, strong inductive bias often wins.

**Key Concept**
Apparent novelties are recognisable variants on the spectrum. Sparse attention is dense attention with most edge weights zeroed. Graph attention is message passing with learned aggregation weights. Each is a trade-off between expressiveness and compute cost.

PART 4

# Architectures

Applying the framework to the canonical families.

Chapters 14 – 21

Chapter 14

# The Motivation Layer: Why Architectures Exist

Every major architecture was invented to solve a specific failure mode of its predecessor. Knowing the problem is as important as knowing the solution.

| Architecture | Problem It Solved | Design Response |
|---|---|---|
| MLP | No function approximator for fixed-size inputs | Stack linear layers with nonlinearities |
| CNN | MLPs on images were quadratically expensive; ignored spatial structure | Shared local kernels exploiting translation equivalence |
| RNN | Variable-length sequences; MLPs need fixed-size inputs | Carry hidden state forward through time |
| LSTM / GRU | Vanilla RNNs could not preserve long-range information | Learned gates control what to keep, forget, or expose |
| Transformer | RNNs sequential; could not parallelise; still struggled long-range | Replace recurrence with direct pairwise attention over all positions |
| ViT | Test whether attention alone suffices for vision given scale | Treat image patches as tokens; apply a standard transformer |
| Diffusion | GANs unstable to train; VAEs blurry | Learn to reverse corruption; generate via iterative denoising |
| MoE | Scaling a single dense model expensive per token | Route each token to a sparse subset of expert modules |

**In Practice**
When reading a new paper, find the sentence that says 'previous methods suffer from X.' That sentence is the motivation. The rest is an engineering response. If you understand X, you can evaluate whether the response actually solves it.

Chapter 15

# Composition Patterns: Encoders, Decoders, and Bottlenecks

We separate **patterns** (the shape of the model) from **mechanisms** (the logic inside each block). The same attention mechanism appears in all three patterns; what differs is information flow direction.

| Pattern | Semantic Role | Mechanism | Examples |
|---|---|---|---|
| Encoder | Understanding and representing input | Bidirectional attention — every entity sees every other | BERT, RoBERTa, DINOv2 |
| Decoder | Generating output token by token | Causal (masked) attention — each entity sees only prior entities | GPT, LLaMA, Mistral |
| Bottleneck | Filtering and compressing a representation | Linear down-projection, optionally followed by up-projection | Adapter layers, cross-modal projections |

## The Three Assembled Configurations

- **Encoder-only:** Full bidirectional attention. Rich input representations. Used for understanding and classification, not generation.
- **Decoder-only:** Causal attention throughout. Generates one token at a time. Used for all forms of language generation and in-context learning.
- **Encoder-decoder:** Encoder builds bidirectional input representation; decoder generates output while cross-attending to encoder states. Used when input and output are both structured sequences.

**Key Concept**
Diagnostic: does the task require reading the full input before producing any output, or producing output token-by-token conditioned on a structured input? Encoder-only for understanding. Decoder-only for generation. Encoder-decoder for structured input-to-output.

Chapter 16

# MLPs: Universal Approximators

```
x → Linear → Nonlinearity → Linear → … → output
```

The MLP applies dense transformations to a fixed-size feature vector. There is no sub-structure: every input dimension can influence every output dimension, but there is no structured communication between parts of the input.

## When MLPs Are the Right Tool

- Tabular data with no inherent spatial, sequential, or relational structure.
- The final task head after an encoder: classification, regression, value prediction.
- The per-entity update sublayer inside transformers, CNNs, and GNNs.

**You've Already Used This**
Tabular ML for click prediction, fraud detection, and churn modelling often uses MLPs as the core model, sometimes combined with embeddings for categorical features.

**Micro-Walkthrough: MLP Forward Pass**
Input: feature vector x = [0.2, -0.8, 0.4] shape [F=3] Layer 1: y1 = ReLU(xW1 + b1) shape [H=8] Layer 2: y2 = ReLU(y1·W2 + b2) shape [H=8] Output: logits = y2·W3 + b3 shape [C=2] Note: no entity interacts with any other entity. Each input is processed independently.

Chapter 17

# CNNs: Local Message Passing on Grids

A CNN is local message passing over a regular grid: 1D for audio or time series, 2D for images, 3D for video or volumetric data.

## EIU View

- **Entity:** Spatial positions. Each carries a vector of channels.
- **Structure:** A regular grid; positions interact only with their local neighbourhood.
- **Interaction:** A shared weight kernel slides across all positions. The same detector applies everywhere — translation equivariance.
- **Update:** Nonlinearity applied to the convolution output.

| Term | Definition |
|---|---|
| Kernel | Small shared weight pattern applied at each position |
| Stride | Step size when sliding the kernel |
| Channel | Feature dimension at each spatial location |
| Feature map | Grid of channel vectors at one layer |
| Receptive field | Input region that can influence one output value |

**Micro-Walkthrough: One Conv Layer**
Input feature map: shape [H=8, W=8, C_in=3] Kernel: shape [3, 3, C_in=3, C_out=16] For each of 8×8=64 positions: dot product of local 3×3×3 patch with kernel weights Output feature map: shape [H=8, W=8, C_out=16] After 4 such layers: 11×11 receptive field. After 8 layers: 23×23. Hierarchy emerges from composition of local operations.

**You've Already Used This**
Face unlock, object detection in self-driving cars, photo enhancement, medical imaging, satellite analysis. CNNs remain dominant wherever spatial locality is a reliable prior.

**Failure Mode**
CNNs trained on natural images can fail dramatically on images with unusual textures or backgrounds not present in training. The translation equivariance prior helps within-distribution but can become a brittle assumption out-of-distribution.

Chapter 18

# RNNs: Sequential Message Passing

```
h_t = update(h_{t-1}, x_t) y_t = readout(h_t)
```

An RNN processes sequences by maintaining a hidden state carried forward through time. The entity is a timestep; the interaction is recurrent state update; the structure is a strictly ordered chain.

## LSTM and GRU: Gated State Machines

Vanilla RNNs overwrite state at every step, making long-range dependency difficult. LSTM and GRU add learned gates — sigmoid functions that decide what fraction to keep, forget, or expose. They significantly extend the range of dependencies the model can capture.

**Micro-Walkthrough: RNN Hidden State Evolution**
Input tokens: ["The", "bank", "by", "the", "river"] h_0 = zeros h_1 = update(h_0, embed("The")) — state begins accumulating context h_2 = update(h_1, embed("bank")) — "bank" integrated; context still ambiguous h_3 = update(h_2, embed("by")) — preposition signals spatial framing h_4 = update(h_3, embed("the")) — article continues spatial context h_5 = update(h_4, embed("river")) — "river" disambiguates; state encodes geographic bank Problem: to reach h_5, "bank" meaning must survive 3 overwrite operations.

## Why Transformers Largely Replaced RNNs

RNNs have an inherent sequential dependency: step t requires step t–1. This prevents parallelising training across timesteps, which is the dominant training cost. Transformers process all positions simultaneously, at the cost of $O(n^2)$ attention.

**Failure Mode**
RNNs forget. Information from early tokens must survive many overwrite steps to influence the final state. LSTM gates help substantially but do not eliminate the fundamental bottleneck for very long sequences.

Chapter 19

# Transformers: Dynamic Message Passing

The transformer is the most general canonical architecture. It is not tied to sequences, grids, or graphs — it operates on any entity set and constructs the interaction graph dynamically at each layer, based on entity content.

## The Transformer Block

```
For each layer: X_attn = Attention(X) # entities exchange information X = LayerNorm(X + X_attn) # residual + normalize X_ff = FFN(X) # each entity updates locally X = LayerNorm(X + X_ff) # residual + normalize
```

## Queries, Keys, and Values

| Term | Role | Computation |
|---|---|---|
| Query | What this entity is looking for | Projects entity into a 'question' space |
| Key | What this entity offers for matching | Projects entity into an 'answer' space |
| Value | What information this entity sends if selected | Projects entity into a 'content' space |

**Micro-Walkthrough: Attention Disambiguates Context**
Sentence A: "The bank approved the loan" Sentence B: "The river overflowed the bank" Both sentences: token "bank" starts with identical embedding e_bank. Sentence A, after attention layer 1: "bank" query matches strongly with keys of "approved", "loan" Attention weights: bank→approved=0.35, bank→loan=0.40, others<0.10 Hidden state h_bank_A shifts toward {financial, institutional} Sentence B, after attention layer 1: "bank" query matches strongly with keys of "river", "overflowed" Attention weights: bank→river=0.42, bank→overflowed=0.31, others<0.10 Hidden state h_bank_B shifts toward {geographic, terrain} Same input token. Same initial embedding. Diverging hidden states after one layer.

## Position Encoding: Why It Is Necessary

Attention is permutation-equivariant without positional information: shuffling the token order changes which tokens are at each position, but no token's representation changes based on its position alone. **Position encodings** — position-specific vectors added to embeddings before processing — give the model access to order information.

## The Quadratic Bottleneck and Responses to It

Dense attention computes $N^2$ pairwise scores for N tokens. At N=512: manageable. At N=32,768: expensive. At N=1,000,000: impractical.

- **Sparse attention** (Longformer, BigBird): each token attends to a local window plus a few global tokens — O(n) cost.
- **Linear attention**: approximates the full matrix with a kernel factorisation — O(n).
- **State space models** (Mamba, S4): replace attention with a linear recurrence that scales linearly and parallelises differently.
- **FlashAttention**: does not reduce asymptotic complexity but computes standard attention faster via IO-aware tiling — now standard in production systems.

**Tensor Trace: Full Transformer Forward Pass**
Input: "The cat sat" (S=3 tokens) Token ids: [464, 3797, 3031] shape [B=1, S=3] After embedding lookup: shape [B=1, S=3, F=768] After position encoding: shape [B=1, S=3, F=768] Q, K, V projections (H=12 heads, d_head=64): each [B=1, H=12, S=3, d=64] Attention scores QK■: shape [B=1, H=12, S=3, S=3] After softmax: shape [B=1, H=12, S=3, S=3] After value aggregation: shape [B=1, H=12, S=3, d=64] After concat + projection: shape [B=1, S=3, F=768] After FFN: shape [B=1, S=3, F=768] After L=12 layers: shape [B=1, S=3, F=768] After logit projection: shape [B=1, S=3, V=50257] Next-token distribution (last position): shape [V=50257]

**You've Already Used This**
Every large language model you have interacted with — ChatGPT, Claude, Gemini, Copilot — is built on transformer blocks. The architecture is also used for image generation, protein structure prediction, and audio synthesis.

**Failure Mode**
Transformers hallucinate because next-token prediction rewards fluent, plausible text, not factually grounded text. A model can assign high probability to a confident-sounding false continuation if it fits the distributional pattern of the context, even if no training example supports the specific claim.

**Test Your Mental Model**
If you removed all attention layers from a transformer and kept only the FFN layers, what capability would disappear? What would remain? Could the model still be trained?

Chapter 20

# Domain Crosswalk: Language, Vision, and Graphs

The EIU framework applies identically across all three major domains. Seeing them side by side makes the universality visible in one place.

| Field | Language (Decoder) | Vision (CNN) | Vision (ViT) | Graph (GNN) |
|---|---|---|---|---|
| Entity | Token | Spatial location | Image patch | Node |
| Representation | Hidden state | Channel vector | Patch embedding | Node embedding |
| Structure | Sequence + causal mask | 2D grid | Set + position encoding | Explicit edges |
| Interaction | Causal self-attention | Local convolution | Full self-attention | Message passing |
| Update | MLP + residual | Nonlinearity + norm | MLP + residual | Node MLP |
| Readout | Logit projection | Pooling + classifier | CLS token + head | Node or graph head |
| Training | Next-token loss | Classification loss | Classification/contrastive | Task-specific |

## What Each Domain Gets Uniquely Right

### *Language*

Causal next-token prediction is a natural self-supervised objective that requires the model to compress everything knowable about context. The causal mask enforces an information constraint that mirrors how language is produced and consumed.

### *Vision with CNNs*

Spatial locality is a reliable prior for natural images. Nearby pixels are almost always more related than distant ones. CNNs exploit this with shared local kernels, making them highly data-efficient relative to vision transformers.

### *Graphs*

Explicit edge structure makes GNNs natural when relational structure is known: molecules (atoms and bonds), knowledge graphs (entities and relations), road networks (intersections and roads). Aggregation must be permutation-invariant because node orderings are arbitrary.

**You've Already Used This**
Language: every text AI. Vision CNNs: photo apps, medical imaging. Vision ViT: DALL-E, Stable Diffusion, modern classifiers. Graphs: drug discovery, recommendation systems, fraud detection.

Chapter 21

# Multimodal Systems

**Key Concept**
A multimodal system has multiple entity types sharing a representation space, connected by cross-attention or projection layers. Once text tokens and image patches live in the same vector space, the architecture behaves like any other transformer.

## Three Fusion Patterns

| Pattern | How Modalities Connect | Trade-off |
|---|---|---|
| Early fusion | All modalities projected to shared space before any layers | Deep cross-modal interaction; more compute |
| Late fusion | Each modality encoded separately; combined before task head | Cheaper; less cross-modal interaction |
| Cross-attention | One modality queries; the other provides keys and values | Asymmetric; common in encoder-decoder systems |

**Micro-Walkthrough: Vision-Language Forward Pass**
Image: 224×224 pixels, split into 14×14=196 patches Patch projection: each 16×16×3 patch → 768-dim vector shape [196, 768] Text: "What is in this image?" → 8 tokens → shape [8, 768] Concatenate along sequence axis: shape [204, 768] Transformer processes all 204 entities; text tokens can attend to patch tokens Readout: last text token hidden state → answer generation

**You've Already Used This**
GPT-4o, Claude with images, Gemini, LLaVA, CLIP, DALL-E. When you upload a photo and ask an AI to describe it, patch embeddings and text token embeddings are being combined through one of these three fusion patterns.

PART 5

# Systems and Scale

How models grow, how knowledge transfers, and how computation executes.

Chapters 22 – 27

Chapter 22

# Scale and Emergent Behavior

Larger models are not just quantitatively better. They sometimes behave qualitatively differently. Understanding both the evidence and the active debates around this prevents over-crediting architectural innovations.

## Scaling Laws

Empirical research has established that model loss decreases smoothly and predictably with model size, training data, and compute budget. The Chinchilla scaling laws (Hoffmann et al., 2022) showed that for a given compute budget, many early large models were substantially undertrained — the optimal strategy is to train a smaller model on more data, not a larger model on fewer tokens.

## Emergent Capabilities: The Debate

Some capabilities appear to switch on sharply above a scale threshold. Three distinct interpretations exist:

- **Genuine phase transitions:** The capability truly does not exist below threshold and appears discretely above it.
- **Smooth capability with a threshold metric:** The capability improves smoothly with scale, but the evaluation metric has a sharp threshold (e.g., exact-match accuracy goes from 0% to non-zero when performance crosses the boundary).
- **Benchmark threshold artifacts:** The apparent emergence is a function of which benchmark is chosen and how it is scored, not a property of the underlying model capability (Schaeffer et al., 2023).

The debate is genuinely unsettled. All three mechanisms may be operating simultaneously for different capabilities.

**Key Concept**
When evaluating a claimed breakthrough: check whether the comparison is compute-matched. Many architectural improvements are scale effects. A fair comparison holds compute constant.

**Test Your Mental Model**
If a new model achieves a 10% improvement on a benchmark over a baseline, what information would you need to determine whether this is an architectural advance, a data improvement, or a scale effect?

Chapter 23

# The Pre-Training Paradigm

The dominant modern workflow is not training a model from scratch for each task. It is pre-training a large model on broad data, then adapting it.

## The Three-Stage Workflow

- **Pre-training:** Train on a massive broad dataset using self-supervised objectives. The model learns general representations. This is the expensive stage.
- **Supervised fine-tuning (SFT):** Continue training on curated instruction-response pairs. The model learns to follow instructions in the format users expect.
- **Alignment (RLHF):** Human raters compare model outputs; a reward model is trained on those preferences; the language model is then optimised against the reward model using reinforcement learning.

## Why Representations Transfer

Features useful for predicting the next word — grammar, facts, discourse structure, reasoning patterns, world knowledge — are also useful for downstream tasks. Pre-training implicitly learns broadly transferable representations. Fine-tuning teaches the model how to deploy those representations for a specific purpose.

> **Key Concept**
> Pre-trained representations are not task-specific. They are general descriptions of the input in high-dimensional space. Fine-tuning teaches the model how to use those descriptions for a specific purpose.

| Approach | Weight Updates? | Data Required | Cost | When to Use |
|---|---|---|---|---|
| Training from scratch | All weights | Very large dataset | Very high | Genuinely novel domain with no relevant pre-trained model |
| Fine-tuning | All or some weights | Moderate task-specific dataset | Medium | Task differs substantially from the base model's training distribution |
| Prompting / ICL | None | A few examples in context | Very low | Task is close to the model's pre-training distribution |

## In-Context Learning

Decoder-only models can learn from examples in their context window without weight updates. Provide three examples of a task in the prompt; the model generalises to new instances. This is qualitatively different from training: no parameters change.

**You've Already Used This**
Every interaction with ChatGPT, Claude, or Gemini involves a model that was pre-trained on broad text, fine-tuned on instruction examples, and aligned with human preferences. The capability you experience is the product of all three stages.

Chapter 24

# Sampling and Decoding: From Logits to Text

The model produces a probability distribution over its vocabulary at each generation step. How you convert that distribution into the next token is the **decoding strategy**. Different strategies produce very different behaviours.

## The Starting Point: Logits

After the final transformer layer, a linear projection maps the hidden state to one score (logit) per vocabulary item. Softmax converts logits to probabilities. Every vocabulary item has a non-zero probability. The question is: which token do we select?

| Strategy | How It Works | Characteristic Behaviour |
|---|---|---|
| Greedy | Always select the highest-probability token | Deterministic; often repetitive or bland |
| Temperature | Divide all logits by T before softmax. T<1 sharpens; T>1 flattens | T=0 ≈ greedy; T>1 = more random and creative |
| Top-k | Sample from the k highest-probability tokens only | Prevents very unlikely tokens; k=1 is greedy |
| Top-p (nucleus) | Sample from the smallest set whose probabilities sum to p | Adapts to distribution shape; p=0.9 is common |
| Beam search | Maintain k best partial sequences; select the overall best | Better for globally coherent output; expensive |

**Micro-Walkthrough: Token Selection with Temperature**
Logits: {cat: 3.2, dog: 2.8, bird: 1.1, fish: 0.4} After softmax (T=1.0): {cat: 0.42, dog: 0.35, bird: 0.14, fish: 0.09} After softmax (T=0.5): {cat: 0.61, dog: 0.34, bird: 0.04, fish: 0.01} [sharper] After softmax (T=2.0): {cat: 0.31, dog: 0.28, bird: 0.23, fish: 0.18} [flatter] Top-p=0.9 at T=1.0: sample from {cat, dog, bird} (cumulative = 0.91) Greedy: always select "cat"

## Hallucination and Decoding

Hallucination — generating fluent but factually wrong content — is partly a training problem (the model learned to be fluent more than factual) and partly a decoding problem (high-temperature sampling may surface confident-sounding completions that were statistically common in training even if factually unsupported).

**Failure Mode**
Repetition loops are a common greedy decoding failure mode: once the model generates a phrase with high probability, the phrase conditions on itself, making repetition even more probable. Repetition penalties are a common engineering fix.

**Test Your Mental Model**
If you wanted a model to generate a creative short story, what temperature and top-p values would you choose? What if you wanted it to extract specific facts from a document? Why might the same values not work well for both?

Chapter 25

# Execution: From Semantics to Kernels

The execution layer turns model semantics into physical computation: tensors become buffers, operations become kernels, graphs become schedules.

| Term | Definition |
|---|---|
| Buffer | A contiguous typed memory block on a device |
| Kernel | A compiled routine executing one operation on hardware |
| Dispatch | The act of launching a kernel |
| Device | CPU, GPU (CUDA/ROCm), Apple Silicon (Metal), or TPU |
| Stride | Memory jumps per step along each tensor axis |
| Fusion | Combining multiple ops into one kernel to reduce memory traffic |
| Fallback | Using a slower generic path when an optimised kernel is unavailable |

## End-to-End Trace: What Happens When You Call matmul

**Execution Trace: out = torch.matmul(A, B)**
1. Python dispatcher receives the call 2. Operator resolution: identifies matmul and its registered implementations 3. Device check: A and B are on CUDA; routes to CUDA backend 4. Shape and dtype validation: [M, K] × [K, N] → output [M, N] 5. Memory allocation: output buffer of shape [M, N] on device 6. Kernel selection: cuBLAS GEMM selected based on shapes and dtype 7. Kernel launch: CUDA kernel executes on GPU streaming multiprocessors 8. Result in output buffer; Python object wraps buffer with shape/stride metadata 9. If autograd enabled: backward node registered recording how to compute dA and dB

## Why Memory Dominates

Many NN operations are memory-bandwidth-limited, not compute-limited. Moving data between high-bandwidth memory (HBM) and compute units is the bottleneck. Fusing operations into one kernel reduces how many times the same large tensor is read from and written to HBM. FlashAttention is the canonical example: it tiles attention to fit in fast on-chip SRAM, dramatically reducing HBM traffic without changing total arithmetic.

Chapter 26

# Runtime Architecture: Eager, Graph, IR, Backend

## Eager Mode

Operations execute immediately as called. Values exist after every line. Standard Python debugging works. PyTorch popularised this for research ergonomics.

## Graph Mode

Computation is captured before execution. The runtime can then optimise, fuse, schedule, and lower the full graph. This is compiler-like and enables optimisations invisible to eager execution — cross-operation fusion, memory planning, device scheduling.

## IR and Backend Lowering

An **IR (intermediate representation)** is the runtime's internal canonical form: a description of operations, shapes, dtypes, and dependencies that is simpler than the user API and backend-independent. **Lowering** converts IR into backend-specific plans: BLAS for CPU, cuBLAS and custom kernels for CUDA, Metal shaders for Apple Silicon.

> **Key Concept**
> High-level semantics should be normalised before backend execution. Backends should not need to understand every frontend convenience.

Chapter 27

# Retrieval-Augmented Generation: A Systems Pattern

RAG is a systems pattern, not an architecture. It combines a parametric model (knowledge in weights) with a non-parametric memory (knowledge in an external index).

## How RAG Works

- A user query is embedded into a vector.
- The vector searches a pre-built index of document embeddings (approximate nearest-neighbour search).
- The top-k retrieved documents are inserted into the model's context.
- The model generates a response conditioned on both the query and retrieved context.

| Property | Parametric Only | RAG System |
|---|---|---|
| Knowledge update | Requires retraining | Update the index |
| Attribution | Difficult — distributed across weights | Possible — retrieved sources are explicit |
| Context | Fixed at training time | Dynamic — any indexed document retrievable |
| Per-query cost | Lower | Higher — embedding search + longer context |

**You've Already Used This**
Perplexity AI, Bing Chat, and enterprise knowledge assistants all use RAG. When an AI response includes citations or references to specific documents, retrieval is almost certainly involved.

**Failure Mode**
RAG fails when retrieval fails: if the relevant document is not in the index, or the query embedding is not close to the document embedding, the model receives no grounding and may hallucinate anyway.

PART 6

# Reference

Tools for applying the framework, glossary, practice maps, and reading.

Chapters 28 – 31

Chapter 28

# How to Read Any New Architecture

When you encounter a new paper or model, apply this checklist before reading the abstract claims.

- How is raw input decomposed into entities?
- What vector or tensor represents each entity, and what is its shape?
- What structure is assumed: sequence, grid, graph, set, or mixed?
- How do entities exchange information? What is the interaction rule?
- What local transformation updates each entity's representation after interaction?
- How many interaction-update cycles are repeated?
- What output head reads the final representations?
- What loss function is used, and what representational pressure does it apply?
- What is the computational bottleneck: memory, matmul, or irregular access?
- **What problem was the previous generation failing at?**
- **Is this encoder-only, decoder-only, or encoder-decoder?**
- **Is the improvement architectural, or is it explained by scale or data?**
- **Does the paper compare against a compute-matched baseline?**

## Architecture Summary Card

| Field | Fill This In |
|---|---|
| Entities | |
| Representation shape | |
| Structure | |
| Interaction | |
| Update | |
| Readout | |
| Loss | |
| Composition | Encoder-only / Decoder-only / Encoder-decoder |
| Motivation | Previous models failed at… |
| Bottleneck | |
| Compute-matched baseline? | Yes / No / Unclear |

Chapter 29

# Glossary, Confusion Pairs, and Terminology Crosswalk

## Core Glossary

**Activation:** A computed internal value during a forward pass.

**Attention:** Weighted aggregation where each entity computes a weighted sum over others' values, with weights from learned query-key similarity.

**Autograd:** Automatic differentiation: records a computation graph during the forward pass and applies the chain rule to compute gradients.

**Backpropagation:** The reverse-mode pass propagating gradients from the loss back through the computation graph.

**Batch:** Independent examples processed together for hardware efficiency. Batch items do not interact.

**Beam search:** A decoding strategy maintaining k best partial sequences and selecting the globally highest-probability completion.

**Bottleneck:** A linear down-projection compressing a representation to lower dimensionality.

**BPE (Byte Pair Encoding):** Tokenisation algorithm iteratively merging the most frequent adjacent character pairs into new vocabulary tokens.

**Channel:** Feature dimension at a spatial location in a CNN feature map.

**Context window:** The set of tokens visible to a language model during one forward pass or generation step.

**Cross-attention:** Attention where queries come from one entity pool and keys/values from another.

**Decoding strategy:** The procedure converting a probability distribution over vocabulary into the next token.

**Distribution shift:** Test data comes from a different distribution than training data, causing degraded model performance.

**Embedding:** Initial vector assigned to a raw object before context-specific processing.

**Emergent capability:** A capability appearing above a scale threshold, absent in smaller models.

**Entity:** The unit represented: token, patch, node, spatial location, or timestep.

**Fine-tuning:** Continued training of a pre-trained model on a smaller task-specific dataset.

**Foundation model:** A model large enough that pre-trained representations are broadly useful across many tasks.

**Gradient:** Local sensitivity of the loss with respect to a parameter.

**Hallucination:** Generating fluent but factually wrong content.

**Hidden state:** A context-dependent internal representation produced after one or more layers.

**In-context learning:** Performing new tasks from examples in the context window without weight updates.

**Inductive bias:** A structural assumption that makes some patterns easier to learn than others.

**KV cache:** Cached key and value tensors from prior generation steps, avoiding recomputation.

**Latent space:** Internal coordinate system in which representations live.

**Layer normalisation:** Rescales each entity's vector to zero mean and unit variance; primarily a training stability mechanism.

**Logit:** Unnormalized score before softmax conversion to probability.

**Loss:** Scalar objective defining training pressure.

**MLP:** Multilayer perceptron: dense linear layers alternating with nonlinear activations.

**Position encoding:** A position-specific vector added to each entity's embedding so the model can distinguish token order.

**Pre-training:** Large-scale training on broad data using a self-supervised objective.

**RAG:** Retrieval-Augmented Generation: combines a parametric language model with a non-parametric external index.

**Representation learning:** Learning to map inputs into geometric spaces where the task becomes computationally easy.

**Residual connection:** Additive skip: x + update(x) instead of replacing x.

**Scaling law:** Empirical relationship: model loss decreases predictably with model size, data, and compute.

**Temperature:** A decoding parameter dividing logits before softmax. Lower T sharpens; higher T flattens.

**Token:** A textual unit produced by a tokenizer: word, subword, or special symbol.

**Top-k sampling:** Sample the next token from the k highest-probability vocabulary items.

**Top-p sampling:** Sample from the smallest set of tokens whose cumulative probability exceeds p.

**Transformer:** Architecture based on attention-based entity interaction, per-entity MLP updates, and residual connections.

## Confusion Pairs

**Embedding vs representation vs hidden state:** An *embedding* is the initial vector for a raw object. A *representation* is any internal vector at any stage. A *hidden state* is context-dependent after layer processing. All embeddings are representations; not all representations are embeddings.

**Parameter vs activation:** *Parameters* are stored in the model and persist across all inputs — they are what training adjusts. *Activations* are computed during a forward pass and differ for every input.

**Layer vs block vs module:** A *layer* is a single learned transformation. A *block* is a group of layers forming one repeating unit. A *module* is a software container; any layer or block is a module.

**Temperature vs top-p vs top-k:** *Temperature* reshapes the entire probability distribution. *Top-k* restricts sampling to k candidates. *Top-p* restricts to the smallest set summing to p. All three are often combined.

**Pre-training vs fine-tuning vs inference:** *Pre-training* is large-scale training from scratch. *Fine-tuning* is continued training from a pre-trained checkpoint. *Inference* is running the model forward to produce outputs, with no parameter updates.

## Cross-Domain Vocabulary Map

| General | Language | Vision (CNN) | Vision (ViT) | Graph |
|---|---|---|---|---|
| Entity | Token | Spatial location | Patch | Node |
| Representation | Hidden state | Channel vector | Patch embedding | Node embedding |
| Interaction | Self-attention | Convolution | Self-attention | Message passing |
| Structure | Sequence | 2D grid | Set + position | Explicit edges |
| Readout | Logit head | Pooling + classifier | CLS + head | Node/graph head |

Chapter 30

# Practice Maps and Closing Synthesis

## Practice Map: Decoder-Only Language Model (GPT / LLaMA)

| Field | Answer |
|---|---|
| Entities | Tokens |
| Representation | [B, S, F] |
| Structure | Sequence + causal mask |
| Interaction | Causal self-attention |
| Update | Per-token MLP + residuals + layer norm |
| Readout | Linear projection to vocabulary logits |
| Loss | Next-token cross-entropy |
| Composition | Decoder-only |
| Motivation | RNNs could not parallelise; transformers process all positions at once |
| Bottleneck | $O(n^2)$ attention; KV cache size at long context |

## Practice Map: CNN Image Classifier

| Field | Answer |
|---|---|
| Entities | Spatial positions |
| Representation | [B, H, W, C] |
| Structure | 2D grid, local neighbourhoods |
| Interaction | Shared local convolution kernel |
| Update | Nonlinearity, normalisation, residual block |
| Readout | Global average pooling + classifier |
| Loss | Cross-entropy |
| Composition | Encoder + task head |
| Motivation | MLPs on images were quadratically expensive; ignored spatial structure |
| Bottleneck | Convolution kernel throughput; memory layout |

## Practice Map: Vision Transformer (ViT)

| Field | Answer |
|---|---|
| Entities | Image patches |
| Representation | [B, num_patches, F] |
| Structure | Set + learned position embeddings |
| Interaction | Full bidirectional self-attention across all patches |
| Update | Per-patch MLP + residuals + layer norm |
| Readout | CLS token + classifier |
| Loss | Classification or contrastive loss |
| Composition | Encoder-only |
| Motivation | Test whether attention alone, given scale, suffices without CNN inductive bias |
| Bottleneck | Attention scales quadratically with number of patches |

## Practice Map: Graph Neural Network

| Field | Answer |
|---|---|
| Entities | Nodes |
| Representation | [num_nodes, F] |
| Structure | Explicit edge topology |
| Interaction | Message passing: each node aggregates from neighbours |
| Update | Node MLP or gated update |
| Readout | Node-level or graph-level pooling head |
| Loss | Task-specific supervised or self-supervised |
| Composition | Encoder + task head |
| Motivation | CNNs and transformers assume fixed spatial or sequential structure |
| Bottleneck | Irregular sparse memory access |

## Practice Map: Encoder-Decoder Language Model (T5 / BART)

| Field | Answer |
|---|---|
| Entities | Source tokens (encoder) + target tokens (decoder) |
| Representation | [B, S, F] for each stack |
| Structure | Encoder: full attention; Decoder: causal + cross-attention to encoder |

| Field | Answer |
|---|---|
| Interaction | Encoder: bidirectional; Decoder: causal + cross-attention |
| Update | MLP + residuals + layer norm in both stacks |
| Readout | Decoder next-token logits |
| Loss | Teacher-forced next-token prediction on target |
| Composition | Encoder-decoder |
| Motivation | Translation needs rich input understanding and sequential output generation |
| Bottleneck | Two full transformer stacks; cross-attention adds cost |

## Final Synthesis

The reader who has completed this book now has four lenses for any architecture: the **What** of what information is represented, the **How** of what math implements it, the **Where** of how hardware executes it, and the **Why** of what problem forced this design to exist.

Most new architectures are modifications of existing choices: a new entity type, a new interaction structure, a new objective, a new composition pattern, or a new execution strategy. When you encounter one for the first time, the question is not 'what is this?' but 'which choices did they change, what problem were they solving, and does the comparison hold compute constant?'

Neural networks learn to organise information in latent representations so that useful behaviour becomes easy to compute — and every architectural decision is ultimately a bet about which organisation of information will make a specific class of useful behaviour easiest to learn, given the data, compute, and hardware available.

Chapter 31

# References and Further Reading


- **Goodfellow, Bengio, and Courville**, *Deep Learning*, 2016. The standard foundational textbook for deep learning concepts and theory.
- **Vaswani et al.**, *Attention Is All You Need*, 2017. Introduced the transformer architecture based entirely on attention mechanisms.
- **He et al.**, *Deep Residual Learning for Image Recognition*, 2015. Introduced residual connections, enabling very deep networks to train reliably.
- **Kingma and Ba**, *Adam: A Method for Stochastic Optimization*, 2014. The Adam optimiser; now the default for most deep learning training.
- **Dosovitskiy et al.**, *An Image Is Worth 16x16 Words*, 2020. Introduced the Vision Transformer; demonstrated attention-only vision at scale.
- **Velickovic et al.**, *Graph Attention Networks*, 2017. Applied masked self-attention to graph-structured data.
- **Ho, Jain, and Abbeel**, *Denoising Diffusion Probabilistic Models*, 2020. The canonical modern diffusion model formulation.
- **Bronstein et al.**, *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*, 2021. Unifies neural architectures through geometric symmetry principles.
- **Kaplan et al.**, *Scaling Laws for Neural Language Models*, 2020. Foundational empirical study of how loss scales with model size, data, and compute.
- **Hoffmann et al.**, *Training Compute-Optimal Large Language Models (Chinchilla)*, 2022. Showed optimal size-to-data ratios; many large models were undertrained.
- **Lewis et al.**, *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020. Introduced the RAG pattern for combining parametric and non-parametric memory.
- **Dao et al.**, *FlashAttention: Fast and Memory-Efficient Exact Attention*, 2022. IO-aware attention via tiling; now standard in production transformer training.
- **Gu and Dao**, *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*, 2023. State space model scaling linearly in sequence length.
- **Schaeffer, Miranda, and Koyejo**, *Are Emergent Abilities of Large Language Models a Mirage?*, 2023. Challenges emergence framing; argues many apparent phase transitions are metric artifacts.
- **Sennrich, Haddow, and Birch**, *Neural Machine Translation of Rare Words with Subword Units*, 2016. Introduced BPE tokenisation for neural machine translation.